



CLICKGEO

CURSOS DE GEOTECNOLOGIAS

Análise Geoespacial em R Part 1

Para realizar uma análise geoespacial com R, a biblioteca mais usada é a `sf` (Simple Features), que é uma das bibliotecas padrão para manipulação e análise de dados geográficos, mas quando falamos de visualização as bibliotecas mais usadas são `ggplot2` junto com `geom_sf`.

```
vetor_pacotes = c(
  "devtools",
  "ggspatial",
  "prettymapr",
  "st",
  "ggplot2",
  "e1071",
  "ggcorrplot",
  "testthat",
  "tidyverse",
  "shiny",
  "leaflet",
  "entropy",
  "sda",
  "fdrtool",
  "corpcor"
)

if (!requireNamespace(vetor_pacotes, quietly = TRUE)) install.packages(vetor_pacotes, dependencies = TRUE)

if (!requireNamespace("ropensci/plotly", quietly = TRUE)) devtools::install_github("ropensci/plotly")

## Skipping install of 'plotly' from a github remote, the SHA1 (16261c33) has not changed since last install
## Use 'force = TRUE' to force installation

if (!requireNamespace("tidyverse/dplyr", quietly = TRUE)) devtools::install_github("tidyverse/dplyr")

## Skipping install of 'dplyr' from a github remote, the SHA1 (d6980222) has not changed since last install
## Use 'force = TRUE' to force installation

if (!requireNamespace('ggplot2', quietly = TRUE)) install.packages('ggplot2')
if (!requireNamespace('sf', quietly = TRUE)) install.packages('sf', dependencies = TRUE)
```

```
if (!requireNamespace("lwgeom", quietly = TRUE)) install.packages("lwgeom", dependencies = TRUE)
if (!requireNamespace("ggmap", quietly = TRUE)) install.packages("ggmap", dependencies = TRUE)
```

Utilizando a linguagem R, você pode eficientemente descompactar arquivos dentro do próprio ambiente de programação através da função `unzip`. Esse processo permite um acesso direto e prático aos dados, sem necessidade de etapas adicionais fora do ambiente de R.

Neste exemplo, vamos trabalhar com os conjuntos de dados dos MUNICÍPIOS e SETORES da Paraíba, ou de qualquer outro estado que seja de seu interesse. Vale ressaltar que os arquivos utilizados aqui são os mesmos empregados na disciplina de Python.

```
library(ggplot2)
library(sf)
```

```
## Linking to GEOS 3.11.2, GDAL 3.8.2, PROJ 9.3.1; sf_use_s2() is TRUE
```

```
library(st)
```

```
## Loading required package: sda
```

```
## Loading required package: entropy
```

```
## Loading required package: corpcor
```

```
## Loading required package: fdrtool
```

```
library(dplyr)
```

```
##
```

```
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##   filter, lag
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
##   intersect, setdiff, setequal, union
```

```
library(leaflet)
```

```
library(lwgeom)
```

```
## Linking to liblwgeom 3.0.0beta1 r16016, GEOS 3.11.2, PROJ 9.3.1
```

```
##
```

```
## Attaching package: 'lwgeom'
```

```
## The following object is masked from 'package:sf':
```

```
##
```

```
##   st_perimeter
```

```
library(purrr)
library(ggmap)
```

```
## i Google's Terms of Service: <https://mapsplatform.google.com>
## Stadia Maps' Terms of Service: <https://stadiamaps.com/terms-of-service/>
## OpenStreetMap's Tile Usage Policy: <https://operations.osmfoundation.org/policies/tiles/>
## i Please cite ggmap if you use it! Use 'citation("ggmap")' for details.
```

```
# Descompactar arquivos shapefile
```

```
unzip("D:/Pos-GeoDataScience/M2-R/dado_pratica/PB_Municipios_2022.zip", exdir = "D:/Pos-GeoDataScience/M2-R/dado_pratica/PB_Municipios_2022")
unzip("D:/Pos-GeoDataScience/M2-R/dado_pratica/PB_Setores_2021.zip", exdir = "D:/Pos-GeoDataScience/M2-R/dado_pratica/PB_Setores_2021")
```

```
# Carregar dados geoespaciais
```

```
municipios <- st_read("D:/Pos-GeoDataScience/M2-R/dado_pratica/PB_Municipios/PB_Municipios_2022.shp")
```

```
## Reading layer 'PB_Municipios_2022' from data source
## 'D:\Pos-GeoDataScience\M2-R\dado_pratica\PB_Municipios\PB_Municipios_2022.shp'
## using driver 'ESRI Shapefile'
## Simple feature collection with 223 features and 4 fields
## Geometry type: MULTIPOLYGON
## Dimension: XY
## Bounding box: xmin: -38.7656 ymin: -8.302955 xmax: -34.79309 ymax: -6.025912
## Geodetic CRS: SIRGAS 2000
```

```
# Converter o objeto sf para data.frame
```

```
municipios_df <- as.data.frame(municipios)
```

```
setores <- st_read("D:/Pos-GeoDataScience/M2-R/dado_pratica/PB_Setores/PB_Setores_2021.shp")
```

```
## Reading layer 'PB_Setores_2021' from data source
## 'D:\Pos-GeoDataScience\M2-R\dado_pratica\PB_Setores\PB_Setores_2021.shp'
## using driver 'ESRI Shapefile'
## Simple feature collection with 9269 features and 12 fields
## Geometry type: MULTIPOLYGON
## Dimension: XY
## Bounding box: xmin: -38.7656 ymin: -8.302955 xmax: -34.79309 ymax: -6.025912
## Geodetic CRS: SIRGAS 2000
```

```
# Converter o objeto sf para data.frame
```

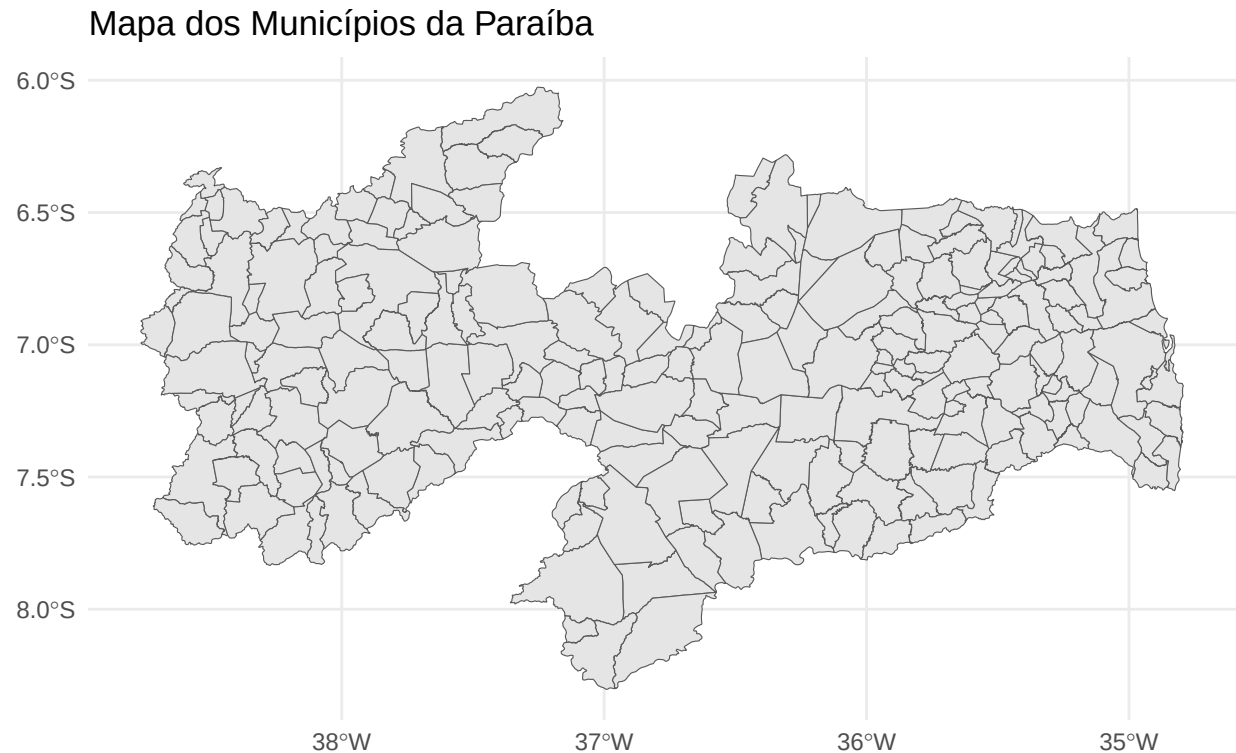
```
setores_df <- as.data.frame(setores)
```

```
# Visualizar as primeiras linhas do DataFrame geoespacial
```

```
head(municipios_df)
```

```
##      CD_MUN      NM_MUN SIGLA_UF AREA_KM2      geometry
## 1 2500106  Água Branca      PB  241.662 MULTIPOLYGON (((-37.61601 -...
## 2 2500205   Aguiar        PB  351.607 MULTIPOLYGON (((-38.27728 -...
## 3 2500304 Alagoa Grande      PB  322.071 MULTIPOLYGON (((-35.51901 -...
## 4 2500403  Alagoa Nova      PB  128.230 MULTIPOLYGON (((-35.76941 -...
## 5 2500502   Alagoinha      PB  111.361 MULTIPOLYGON (((-35.55921 -...
## 6 2500536   Alcantil      PB  309.896 MULTIPOLYGON (((-36.03541 -...
```

```
# Exibir o mapa dos municípios
ggplot(municipios) +
  geom_sf() +
  theme_minimal() +
  ggtitle("Mapa dos Municípios da Paraíba")
```



Manipulação de Dados Geospaciais Vetoriais A manipulação de dados geospaciais vetoriais em R pode ser efetuada com a ajuda de pacotes como **sf** e **rgeos**. Esses são os pacotes mais usados para essas atividades, os quais oferecem funcionalidades semelhantes às bibliotecas Python GeoPandas e Shapely.

Para começar, importamos os dados geospaciais e selecionamos um município específico, similarmente ao que foi feito no exemplo de Python.

```
# Converter o objeto sf para data.frame
municipios_df <- as.data.frame(municipios)
```

```
head(municipios_df)
```

##	CD_MUN	NM_MUN	SIGLA_UF	AREA_KM2	geometry
## 1	2500106	Água Branca	PB	241.662	MULTIPOLYGON (((-37.61601 -...
## 2	2500205	Aguiar	PB	351.607	MULTIPOLYGON (((-38.27728 -...
## 3	2500304	Alagoa Grande	PB	322.071	MULTIPOLYGON (((-35.51901 -...
## 4	2500403	Alagoa Nova	PB	128.230	MULTIPOLYGON (((-35.76941 -...
## 5	2500502	Alagoinha	PB	111.361	MULTIPOLYGON (((-35.55921 -...
## 6	2500536	Alcantil	PB	309.896	MULTIPOLYGON (((-36.03541 -...

```

# Converter o objeto data.frame para sf
municipios_sf <- st_as_sf(municipios_df, sf_column_name = "geometry")

# Selecionar um município específico
municipio_especifico <- municipios_sf %>% filter(municipios_sf$NM_MUN == 'João Pessoa')

# Plotar o município selecionado
plot(st_geometry(municipio_especifico))
title('Mapa do Município João Pessoa')

```

Mapa do Município João Pessoa



Simplificação e Generalização Em R, as operações de simplificação e generalização são realizadas principalmente com a função `st_simplify` do pacote `sf`.

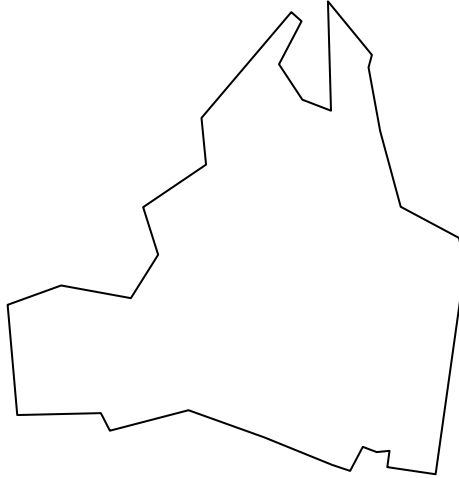
```

# Simplificar a geometria do município
municipio_simplificado <- st_simplify(municipio_especifico, dTolerance = 500)

# Plotar o município simplificado
plot(st_geometry(municipio_simplificado))
title('Mapa do Município Simplificado')

```

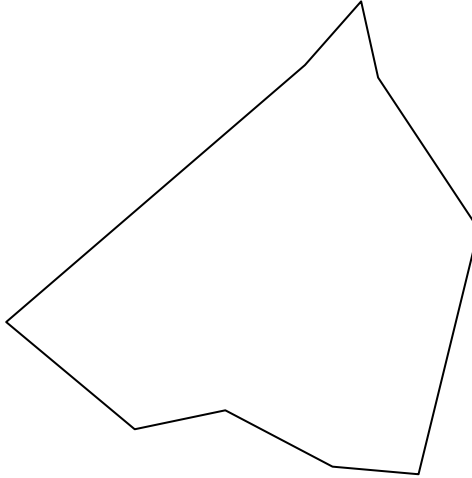
Mapa do Município Simplificado



```
# Generalizar a geometria do município  
municipio_generalizado <- st_simplify(municipio_especifico, dTolerance = 3000)
```

```
# Plotar o município generalizado  
plot(st_geometry(municipio_generalizado))  
title('Mapa do Município Generalizado')
```

Mapa do Município Generalizado



Reprojeção de Dados Geoespaciais em R

A reprojeção de dados geoespaciais para diferentes sistemas de coordenadas é uma tarefa comum em geo-processamento, e pode ser realizada em R usando a função `st_transform`. Essa função permite alterar o Sistema de Referência de Coordenadas (SRC) de um conjunto de dados espaciais.

```
# Reprojetar o GeoDataFrame para um CRS projetado (exemplo: EPSG:3857)
municipio_projetado <- st_transform(municipio_especifico, crs = 3857)
setores_projetado <- st_transform(setores, crs = 3857)

# Verificar o sistema de coordenadas resultante
print(st_crs(municipio_especifico))
```

```
## Coordinate Reference System:
##   User input: SIRGAS 2000
##   wkt:
##   GEOGCRS["SIRGAS 2000",
##     DATUM["Sistema de Referencia Geocentrico para las AmericaS 2000",
##       ELLIPSOID["GRS 1980",6378137,298.257222101,
##         LENGTHUNIT["metre",1]],
##       PRIMEM["Greenwich",0,
##         ANGLEUNIT["degree",0.0174532925199433]],
##     CS[ellipsoidal,2],
##     AXIS["geodetic latitude (Lat)",north,
##       ORDER[1],
```

```

##          ANGLEUNIT["degree",0.0174532925199433]],
##          AXIS["geodetic longitude (Lon)",east,
##              ORDER[2],
##              ANGLEUNIT["degree",0.0174532925199433]],
##      USAGE[
##          SCOPE["Horizontal component of 3D system."],
##          AREA["Latin America - Central America and South America - onshore and offshore. Brazil - onshore"],
##          BBOX[-59.87,-122.19,32.72,-25.28]],
##      ID["EPSG",4674]]

```

```
print(st_crs(municipio_projetado))
```

```

## Coordinate Reference System:
##   User input: EPSG:3857
##   wkt:
## PROJCRS["WGS 84 / Pseudo-Mercator",
##     BASEGEOGCRS["WGS 84",
##       ENSEMBLE["World Geodetic System 1984 ensemble",
##         MEMBER["World Geodetic System 1984 (Transit)"],
##         MEMBER["World Geodetic System 1984 (G730)"],
##         MEMBER["World Geodetic System 1984 (G873)"],
##         MEMBER["World Geodetic System 1984 (G1150)"],
##         MEMBER["World Geodetic System 1984 (G1674)"],
##         MEMBER["World Geodetic System 1984 (G1762)"],
##         MEMBER["World Geodetic System 1984 (G2139)"],
##       ELLIPSOID["WGS 84",6378137,298.257223563,
##         LENGTHUNIT["metre",1]],
##       ENSEMBLEACCURACY[2.0]],
##     PRIMEM["Greenwich",0,
##       ANGLEUNIT["degree",0.0174532925199433]],
##     ID["EPSG",4326]],
##   CONVERSION["Popular Visualisation Pseudo-Mercator",
##     METHOD["Popular Visualisation Pseudo Mercator",
##       ID["EPSG",1024]],
##     PARAMETER["Latitude of natural origin",0,
##       ANGLEUNIT["degree",0.0174532925199433],
##       ID["EPSG",8801]],
##     PARAMETER["Longitude of natural origin",0,
##       ANGLEUNIT["degree",0.0174532925199433],
##       ID["EPSG",8802]],
##     PARAMETER["False easting",0,
##       LENGTHUNIT["metre",1],
##       ID["EPSG",8806]],
##     PARAMETER["False northing",0,
##       LENGTHUNIT["metre",1],
##       ID["EPSG",8807]]],
##   CS[Cartesian,2],
##     AXIS["easting (X)",east,
##       ORDER[1],
##       LENGTHUNIT["metre",1]],
##     AXIS["northing (Y)",north,
##       ORDER[2],
##       LENGTHUNIT["metre",1]],
##   USAGE[

```



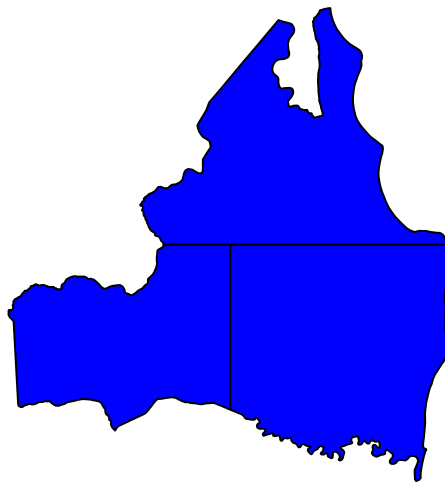
```
##      SCOPE["Web mapping and visualisation."],  
##      AREA["World between 85.06°S and 85.06°N."],  
##      BBOX[-85.06,-180,85.06,180]],  
##      ID["EPSG",3857]]
```

Divisão de Geometrias

Em análises geoestatísticas, frequentemente surge a necessidade de subdividir grandes regiões em parcelas menores para uma investigação mais detalhada. Existem múltiplas abordagens para realizar esta tarefa; no entanto, ao empregar a linguagem R, podemos efetuar esta subdivisão de maneira eficiente e precisa através da função `st_subdivide`. Esta função requer apenas que se especifique a área a ser subdividida e o número máximo de vértices por parcela.

```
sub <- st_subdivide(municipio_projetado, 1000)  
  
plot(sub$geometry, col='blue', main = 'Subdivisão',)
```

Subdivisão



União de Geometrias

Outro processo importante nas análises geoestatísticas é a união ou dissolve de polígonos e regiões de interesse, procedimento oposto à subdivisão. Para efetivar essa união utiliza-se a função `st_union`. Ela permite a fusão eficiente de múltiplas geometrias em uma única entidade coesa. Para assegurar a integração total dos polígonos, especialmente quando há pequenas lacunas ou sobreposições entre eles, recomenda-se o uso de `st_buffer` para aplicar um pequeno buffer antes da união.

```

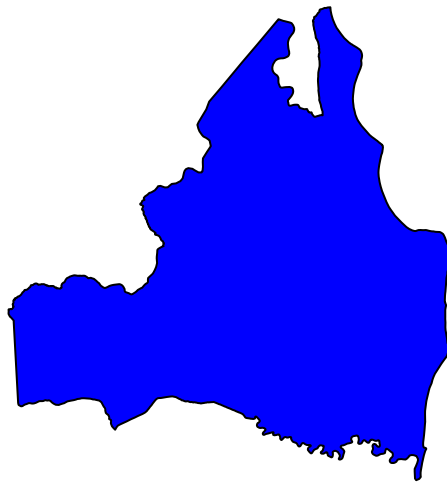
sub <- st_buffer(sub, 0.5)

# Realizar união das geometrias divididas
municipios_united <- st_union(sub)
municipios_united <- st_buffer(sub, -0.5)

# Plotar o resultado da união
plot(municipios_united$geometry, main = 'União das Subdivisões', col='blue')

```

União das Subdivisões



Transformações Geométricas (Rotação, Escalonamento e Translação)

Já sobre aos processos de transformações geométricas como: rotação, escalonamento e translação, a biblioteca `sf` oferece suporte a essas operações, facilitando a manipulação espacial dos dados. No entanto, há espaço para personalização e expansão dessas capacidades por meio do desenvolvimento de funções próprias. Permitindo a implementação de algoritmos específicos que podem atender a necessidades únicas da manipulação geométrica.

```

# Função para rotação coordenadas x, y
rotate_coords <- function(x, y, angle_degrees) {
  angle_radians <- pi * angle_degrees / 180
  x_rotated <- cos(angle_radians) * x - sin(angle_radians) * y
  y_rotated <- sin(angle_radians) * x + cos(angle_radians) * y
  return(list(x = x_rotated, y = y_rotated))
}

```

```

# Função para aplicar mudança de escala a coordenadas
scale_coords <- function(x, y, scale_x, scale_y) {
  x_scaled <- x * scale_x
  y_scaled <- y * scale_y
  return(list(x = x_scaled, y = y_scaled))
}

# Função para transladar coordenadas x, y
translate_coords <- function(x, y, shift_x, shift_y) {
  x_translated <- x + shift_x
  y_translated <- y + shift_y
  return(list(x = x_translated, y = y_translated))
}

# Função simplificada para escalar
scale_multipolygon <- function(geometry, scale_factor) {
  # Processando cada geometria individualmente
  scaled_geom <- lapply(st_geometry(geometry), function(geom) {
    # Aplicando escala a cada ponto em cada polígono
    geom <- lapply(geom, function(polygon) {
      lapply(polygon, function(ring) {
        coords <- scale_coords(ring[,1], ring[,2], scale_factor, scale_factor)
        cbind(coords$x, coords$y)
      })
    })
    st_multipolygon(geom)
  })

  st_sfc(scaled_geom, crs = st_crs(geometry))
}

# Função de rotação para geometrias MULTIPOLYGON
rotate_multipolygon <- function(geometry, angle_degrees) {
  # Processando cada geometria individualmente
  rotated_geom <- lapply(st_geometry(geometry), function(geom) {
    # Aplicando rotação a cada ponto em cada polígono
    geom <- lapply(geom, function(polygon) {
      lapply(polygon, function(ring) {
        coords <- rotate_coords(ring[,1], ring[,2], angle_degrees)
        cbind(coords$x, coords$y)
      })
    })
    st_multipolygon(geom)
  })

  st_sfc(rotated_geom, crs = st_crs(geometry))
}

# Função de translação para geometrias MULTIPOLYGON
translate_multipolygon <- function(geometry, shift_x, shift_y) {

```

```

# Processando cada geometria individualmente
translated_geom <- lapply(st_geometry(geometry), function(geom) {
  # Aplicando translação a cada ponto em cada polígono
  geom <- lapply(geom, function(polygon) {
    lapply(polygon, function(ring) {
      coords <- translate_coords(ring[,1], ring[,2], shift_x, shift_y)
      cbind(coords$x, coords$y)
    })
  })
  st_multipolygon(geom)
})

st_sfc(translated_geom, crs = st_crs(geometry))
}

```

```

# Aplicando rotação
municipio_t <- rotate_multipolygon(municipio_projetado, 45)

# Aplicando mudança de escala
municipio_t <- scale_multipolygon(municipio_t, 2)

# Aplicando translação
municipio_t <- translate_multipolygon(municipio_t, 10, 5)

# Plotando o resultado
plot(st_geometry(municipio_t))

```



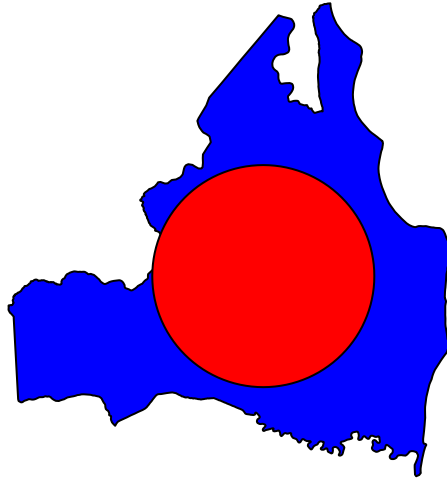
Buffer

O Buffer refere-se à criação de uma área perimetral ao redor de um elemento geométrico seja: ponto, linha ou polígono e definida por uma distância específica. Na prática, ao aplicar um buffer em um mapa, ao redor de um ponto, por exemplo, estamos delineando uma zona de influência ou interesse que circunda esse ponto. Esse procedimento é utilizado para identificar e analisar regiões que estão dentro de um alcance determinado, facilitando estudos de proximidade e impactos.

```
# Criar o centroide do município
centro_municipio <- st_centroid(municipio_projetado$geometry)

# Criar um buffer de 5 km ao redor do centroide
buffer_centro <- st_buffer(centro_municipio, dist = 5000)

# Plotar o buffer
plot(municipio_projetado$geometry, col = 'Blue')
plot(buffer_centro, add = TRUE, col = 'red')
```

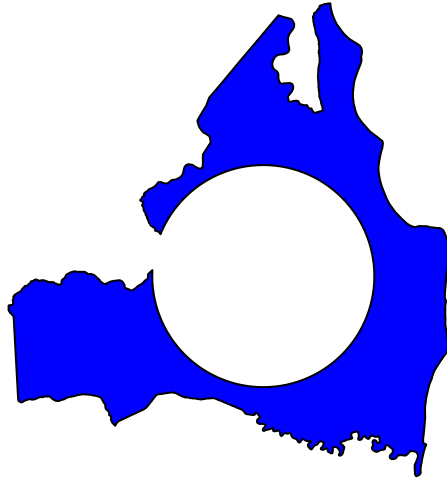


Diferença A diferença geométrica permite a subtração de uma forma geométrica de outra, e tem o papel de oferecer insights valiosos em diversas aplicações espaciais. É possível, por exemplo, remover a área de um buffer de um polígono de maior dimensão, identificando uma área fora da zona de atuação.

```
# Garantir que o buffer e o município estão no mesmo CRS
buffer_centro <- st_transform(buffer_centro, st_crs(municipio_projetado$geometry))

# Criar a diferença entre o município e o buffer
diferenca <- st_difference(municipio_projetado$geometry, buffer_centro)

# Plotar a diferença
plot(diferenca, col = 'blue')
```



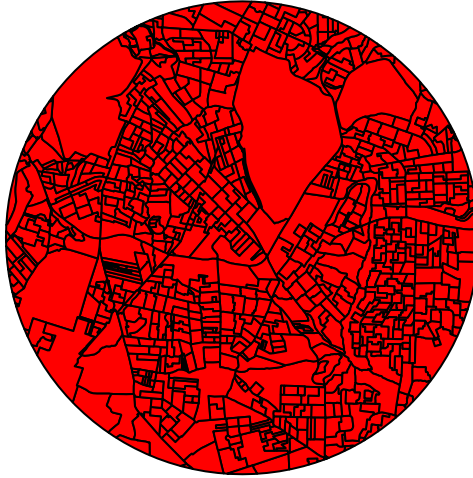
Recorte ou Intersecção

Já intersecção é uma operação fundamental em análises geoespaciais, usada para identificar a sobreposição entre dois conjuntos de dados espaciais. Por exemplo, pode-se querer identificar quais setores censitários estão dentro de um buffer específico.

```
# Projetar os setores para o mesmo CRS do buffer
setores_projetado <- st_transform(setores_projetado$geometry, st_crs(buffer_centro))

# Realizar a intersecção entre setores e buffer
setores_interseccao <- st_intersection(setores_projetado, buffer_centro)

# Plotar os setores que intersectam com o buffer
plot(setores_interseccao, col = 'red')
```



Visualização

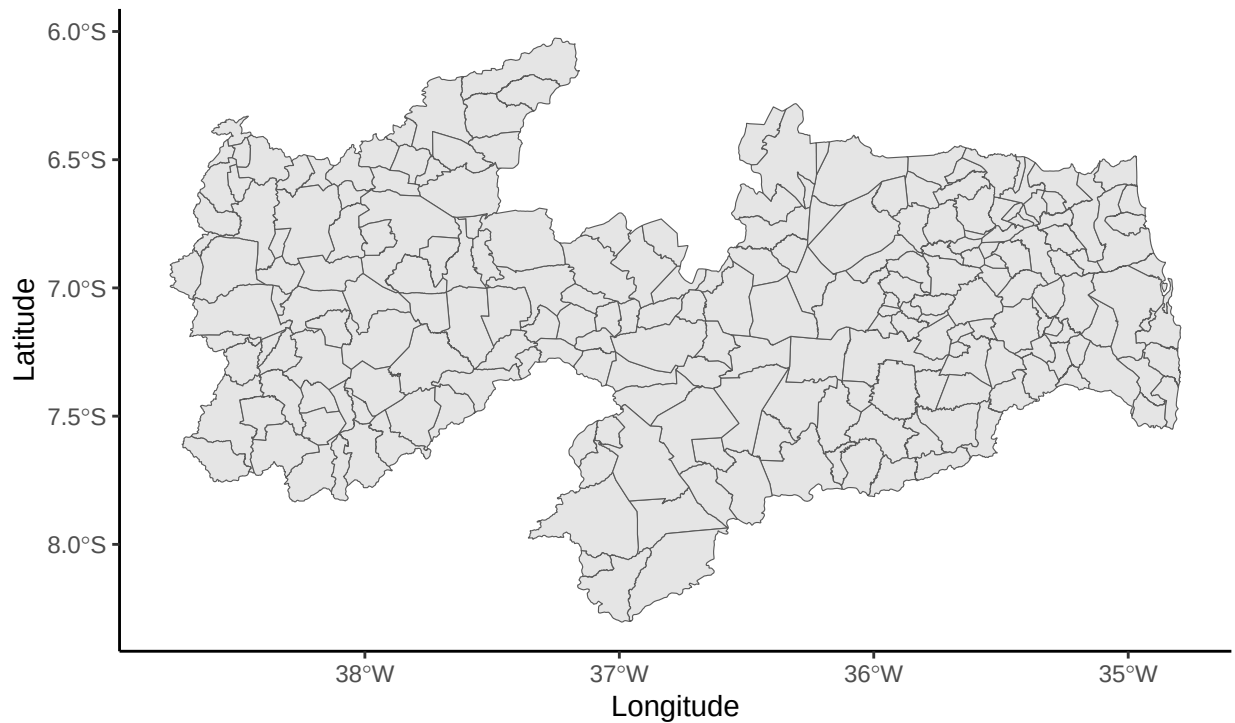
Visualização Básica com ggplot2

O ggplot2 é uma biblioteca poderosa e versátil para visualização de dados em R, amplamente reconhecida por sua capacidade de gerar gráficos esteticamente agradáveis e tecnicamente precisos, incluindo mapas e visualizações baseadas em dados geoespaciais. Esta biblioteca se destaca pela sua facilidade de uso e flexibilidade, permitindo aos usuários criar uma vasta gama de visualizações complexas com comandos relativamente simples. Além disso, o ggplot2 oferece uma ampla seleção de temas de visualização, que podem ser personalizados para atender às necessidades específicas de qualquer projeto de análise de dados. Para explorar as opções de temas disponíveis e encontrar inspiração para seus próprios gráficos, confira a documentação oficial sobre temas no ggplot2 através deste link: [GGPLO2](#)

```
library(ggspatial)
```

```
# Criar plotagem básica com ggplot2
ggplot(data = municipios) +
  geom_sf() +
  labs(title = "Visualização Básica de Dados Geoespaciais",
       x = "Longitude",
       y = "Latitude") +
  theme_classic()
```


Visualização Básica de Dados Geoespaciais

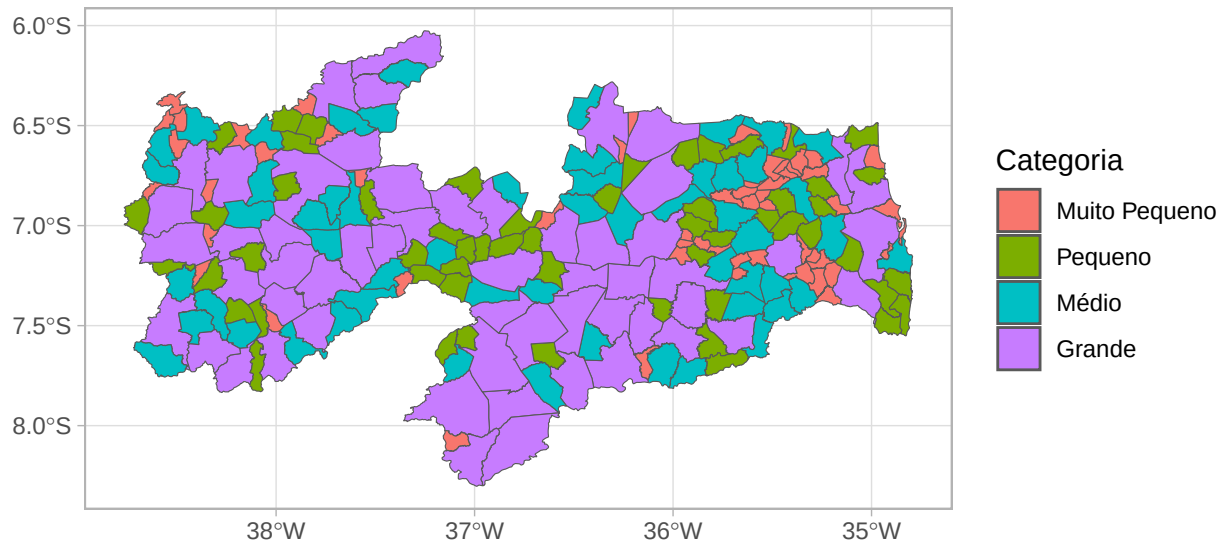


Utilizando o pacote `ggplot2` é possível não apenas visualizar dados, mas também categorizar e destacar variações dentro de um conjunto de dados geoespaciais de forma intuitiva e visualmente atraente. Um exemplo prático disso é a categorização de municípios com base na sua área total, seguida da criação de um mapa colorido que reflete essas categorias, podendo criar uma plotagem com base nisso.

```
# Categorizar municípios baseado na área
municipios$categoria <- cut(municipios$AREA_KM2,
                             breaks = quantile(municipios$AREA_KM2, probs = 0:4/4),
                             labels = c("Muito Pequeno", "Pequeno", "Médio", "Grande"),
                             include.lowest = TRUE)

# Criar plotagem com cores por categoria de área
ggplot(data = municipios) +
  geom_sf(aes(fill = categoria)) +
  labs(title = "Mapa com Cores por Categoria de Área",
       fill = "Categoria") +
  theme_light()
```

Mapa com Cores por Categoria de Área



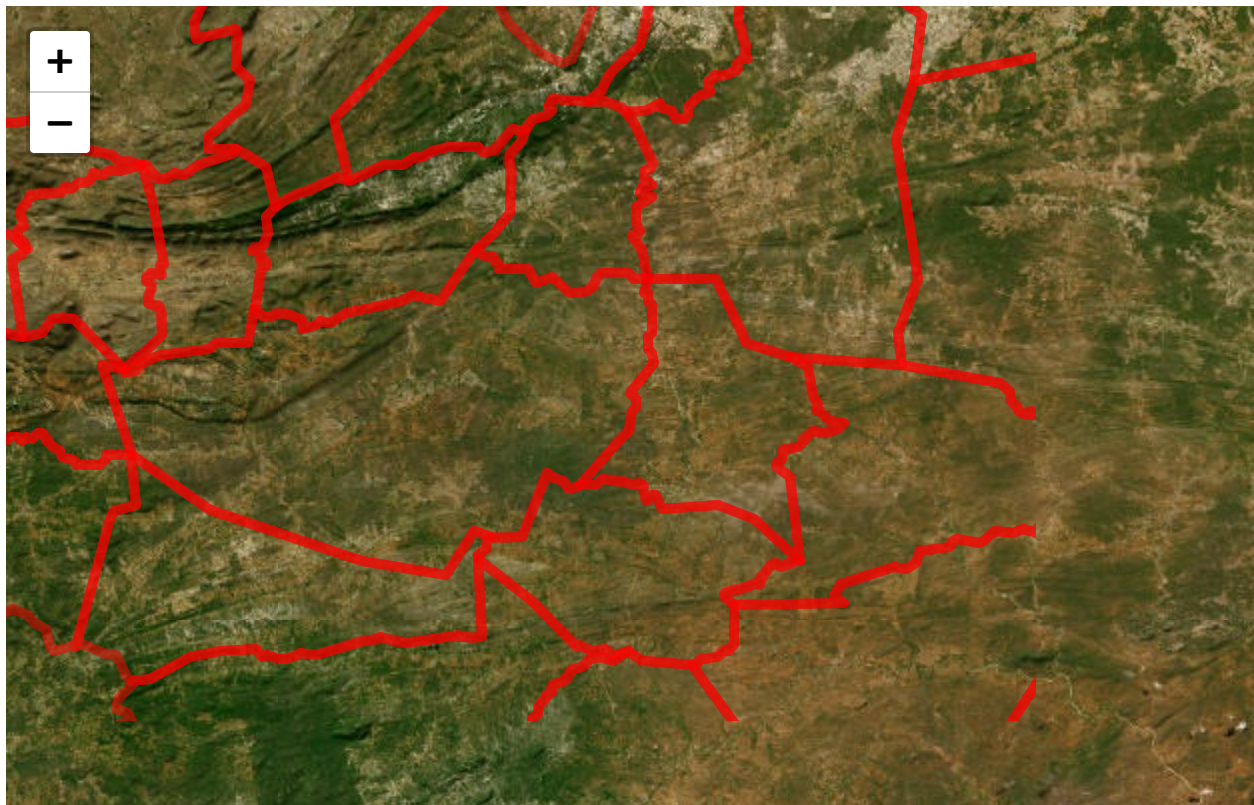
O pacote `leaflet`, em combinação com `sf`, viabiliza a incorporação de basemaps, incluindo imagens de satélite e OpenStreetMap, aos nossos mapas, resultando em uma experiência interativa semelhante ao que observamos com ferramentas em Python.

```
# Criar um mapa usando leaflet
mapa <- leaflet() %>%
  addProviderTiles(providers$Esri.WorldImagery) %>%
  addPolygons(data = municipios, color = "#FF0000", fill = FALSE) %>%
  setView(lng = mean(st_bbox(municipios)[c("xmin", "xmax")]),
    lat = mean(st_bbox(municipios)[c("ymin", "ymax")]),
    zoom = 10)
```

```
## Warning: sf layer has inconsistent datum (+proj=longlat +ellps=GRS80 +towgs84=0,0,0,0,0,0,0 +no_defs)
## Need '+proj=longlat +datum=WGS84'
```

```
# Exibir o mapa
mapa
```

```
## Google Chrome was not found. Try setting the 'CHROMOTE_CHROME' environment variable to the executable
```



Leaflet | Tiles © Esri — Source: Esri, i-cubed, USDA, USGS, AEX, GeoEye, Getmapping, Aerogrid, IGN, IGP, UPR-EGP, and the GIS User Community